

UO Assistant 0.7.0

Scripting and automation reference

Unsigned Windows development preview. Settings belong to your selected shard profile. Start actions explicitly and use Stop all to end assistant work.

Eight categories, with tabs for related tools

Choose a category on the left, then a tab across the top. A second row appears when a tab has related tools. Save and Run buttons stay above the scrolling page, and Stop all is always at the top of the window.

Home: Overview, activity and journal, timers.

Character: Health and pets, targets and saved objects, equipment, combat and cooldowns.

Items: Bags and search, loot, moving and restocking, vendors and use-once items.

Tasks: Your task library, Simple steps, JavaScript, recorded actions, workflows, crafting, monitors and debugging.

Map & travel: Map and markers, walking and waypoints, runebooks, team map and recovery.

Controls: Hotkeys, keyboard and mouse bindings, action bars, display and sound.

Recording: Video capture, replay and recording settings. Action recording for macros is under Tasks.

Settings: Profiles, play styles, appearance, backups and diagnostics.

Find a tool accepts familiar words such as bandage, trees, bags and hotkey. Favorites retain your saved tools. Show advanced tools reveals specialist editors, inspectors and diagnostics. Opening an advanced tool from a direct link reveals it automatically. All previous features remain available.

Getting started with the new layout

Start with Simple Scripts

Open Tasks / Editor / Simple steps, or choose Create a Simple Script on Home / Overview. Pick a starter recipe, then Use this recipe. Read the steps. Check steps checks the whole script without running anything. Run these steps runs the current editor, even before you save it. Stop ends the active run.

The Add a step menus write commands for you. Choose an action, fill in the names or numbers, inspect the preview, then Add this step. Steps are appended in order. You can edit the text, use Ctrl+Space to insert a command example, or wrap all your steps in a finite repeat.

Save script keeps your work in the character library. Export these steps creates a .uosimple file. Import .uosimple or .js files on Script tools. Existing JavaScript scripts remain JavaScript. Make an advanced JavaScript copy saves the translated code and helper functions as a separate script; your Simple source stays intact.

```
player.cast "flamestrike" on last
wait 1 second
player.cast "heal" on self
```

Give objects useful names

In Simple Scripts, enter a name under Give an object a name and choose Choose an object in game. Click the axe, bag, bandages or other object in the game. Use saved "my axe" or saved "wood bag" to refer to that exact object. A name already attached to a different object is not silently replaced; edit it on Targeting or choose another name.

A quoted item name searches the exact loaded item name and ignores capitals. It does not use a saved-object name unless you write saved. A graphic selects an artwork number; hue, colour or color narrows its colour. A serial identifies an exact object. Missing names, duplicate matches and unopened bags produce an explanation instead of an arbitrary selection.

Spell and skill names come from the connected client. Spaces, capitals, hyphens and underscores are ignored, so Flame Strike and flamestrike match. Unknown or ambiguous names stop the script. Shard-specific additions must appear in that client catalogue.

```
items.use saved "bandages" on self
items.use graphic 0x0F49 hue 7
items.equip saved "my cloak" layer "cloak"
```

Targets and waiting

Use on self, on last or on saved "name" after a spell or item use. The script waits for the fresh target cursor automatically. You can also put target.self, target.last or target saved "name" on a separate line. A five-second target timeout stops the script; it does not cast or use the item repeatedly.

A target from a scanned map tile uses its coordinates and original graphic. The exact tile is checked again before targeting. A manual action, changed cursor, changed character or stopped run invalidates the old authority. Game actions remain subject to the shard and current loaded state.

```
player.cast "Greater Heal"
target.self
```

Put items into containers

Open the source and destination bags first. Save a memorable name for the destination. Searches cover the direct contents of the chosen open bag, not nested bags. The default source is your backpack. Matching ordinary stacks can be moved; containers themselves are not moved by this command.

amount all moves every matching stack that was found in the direct source. A number means a total amount across stacks. An insufficient total is refused before moving anything. Every requested move must be observed in the inventory before the next move. A changed item, protected item or unconfirmed move stops the script and asks you to check the bags.

There is no automatic banking, opening of nested bags or dropping on the ground. Use a smaller or more specific selection when the loaded inventory is incomplete.

```
items.move graphic 0x1BDD to saved "wood bag" amount all from backpack
items.move graphic 0x0E21 to saved "supplies" amount 10 from backpack
```

Scan trees, caves, items and windows

scan.trees, scan.caves and scan.tiles collect every bounded page around your starting point. Results are nearest first. The default radius is 12 tiles, maximum 18. Unloaded cells, truncated pages or a broken cursor stop the scan. Custom artwork may require scan.tiles graphic instead of tree/cave classification.

Use as to give the result list a name, then show to inspect up to twelve results in script output. Found item rows include name, graphic, object and amount; tile rows include coordinates. Window rows include visible text and available button details. Larger lists are explicitly shortened for display; loops still use the whole bounded list.

General gump scanning is read-only and searches visible window text by substring, or graphics. Editable fields are excluded. Replies to server windows use a separate gumps.wait followed by gumps.press. An inspection result does not itself authorize a reply.

```
scan.trees within 12 as trees
show trees
scan.caves within 8 as caves
show caves
items.find graphic 0x1BDD in backpack as logs
show logs
gumps.find "craft" as windows
show windows
```

Walk and visit results

walk.to and run.to accept X and Y with optional z height, or a tile from a scan. Omitted height uses the current player height. within is the stopping distance from zero to three tiles (default one). timeout defaults to ten seconds, maximum thirty. Movement is limited to nearby loaded locations within 32 tiles per move.

The script waits until arrival is observed. An unreachable destination stops pathfinding and the script. A scan can identify a tree that cannot be reached; finding a tile is not proof it is walkable or harvestable.

```
scan.trees within 8 as trees
for each tree in trees
    walk.to tree within 2
end
```

Chop nearby trees on Invicta

Use the Chop nearby trees - Invicta starter recipe. Equip your axe, save it as lumberAxe using Choose an object, and dismount. The recipe selects a 30-minute maximum run. The gathering builder also selects that limit when adding its step. Start within the loaded forest area you want to harvest.

The one command scans a fixed radius of up to twelve tiles, walks between tree positions, and uses the equipped axe until the server says that position has no wood left. It stops for damage, low health, weight reserve, a changed or broken tool, manual actions, unknown journal replies or finite run/operation limits.

Unreachable, blocked and repeatedly failing trees are counted as skipped, never as depleted. The final output distinguishes an area finished from a pass with skips. Logs stay in your backpack. If the run stops for weight, unload them and run again. This recipe recognises Invicta messages; other shards may need an advanced-script adaptation. Automated synthetic checks do not replace a live shard test.

```
trees.chop nearby using "lumberAxe" within 12
```

Repeat and make decisions

repeat runs from one to one hundred times. for each visits the items or tiles returned by an earlier find or scan. end closes each repeat, for each or if block. else is optional. Indentation helps readability. At most eight blocks may be nested.

Conditions support player.health, player.mana, player.stamina and player.weight, with below, above or equals and an optional percent. A percentage needs a known positive maximum. Other conditions are items.have, journal.contains, player.dead and player.mounted.

break finishes the current loop; continue skips its current iteration; stop finishes the whole script. Run-time and engine-operation budgets apply in addition to finite loops. Choose 1, 5 or 30 minutes in the beginner editor. Imported custom run limits are retained until you change the choice.

```
repeat 3 times
    if player.health below 50 percent
        player.cast "Heal" on self
    else
        player.skill "Hiding"
    end
    wait 10 seconds
end
```

Windows and journal messages

gumps.wait accepts text or a server serial, and waits for exactly one complete matching server window. gumps.press takes a button number or the exact button label if the client supplies one. Missing or ambiguous labels require a number from the inspector. The window must remain unchanged after the wait. Menus use menus.wait and menus.choose.

prompt.reply and entry.reply wait for their corresponding text dialog before replying. General window search cannot click arbitrary local client windows. Page changes, multiple matches and changing server windows may require a fresh wait.

journal.clear marks the current sequence without erasing the game journal. journal.wait and journal.contains use new system messages after the script starts or that marker. Chat from other players does not satisfy these tests. log and its say alias write to assistant output, not game chat.

```
gumps.wait "craft" timeout 5 seconds
gumps.press button 1
journal.clear
items.use saved "my item"
journal.wait "You put some" timeout 10 seconds
```

When something does not work

Check steps validates the entire source before any game action. Runtime errors identify the original Simple Script line. A valid script still needs the correct shard, loaded objects, connected character and available spell/skill catalogue. Use Choose an object when item names are missing or several objects match.

An action being requested does not prove that the shard accepted its effect. Casting is followed by a cursor wait when targeting is requested. Movement waits for observed arrival; item transfers wait for observed changes. Journal-driven recipes stop when their expected replies do not arrive.

Simple Script is an additional language, not a replacement for JavaScript. All 48 advanced assistant API methods, legacy import, hotkeys, observers and debugging remain available. Source-line debugging and the offline bench are for JavaScript; the beginner editor offers validation and the shared action trace. Existing JavaScript source is not automatically rewritten.

Player commands

Cast a spell

```
player.cast "Heal" on self
```

Choose a spell name. Use on self, on last or on saved "name". Targeting waits automatically.

Use a skill

```
player.skill "Hiding"
```

Spell and skill names ignore spaces, capitals, hyphens and underscores.

Open backpack

```
player.backpack
```

Opens your backpack. Open inner bags yourself before searching or moving their contents.

Open map

```
player.map
```

Opens the game map.

Dismount

```
player.dismount
```

Gets off your mount.

War mode

```
player.war on
```

Choose on or off.

Attack last target

```
player.attack last
```

Saved friends, party members and yourself remain protected.

Primary ability

```
player.primary
```

Requests your primary weapon ability.

Secondary ability

```
player.secondary
```

Requests your secondary weapon ability.

Toggle flight

```
player.fly
```

Toggles flight when supported by your character and shard.

Targets commands

Target yourself

```
target.self
```

Waits up to five seconds for a fresh target cursor, then targets you.

Target last

```
target.last
```

Uses your last target after waiting for a fresh cursor.

Target saved object

```
target saved "supplies"
```

Use Choose an object to give something a memorable saved name.

Target ground

```
target.ground 100 100 0 graphic 0
```

X, Y, Z and optional graphic. A scanned tile can instead be targeted by its loop name.

Wait for cursor

```
target.wait 5 seconds
```

Normally automatic when targeting. An expired wait stops the script.

Cancel cursor

```
target.cancel
```

Cancels the currently available target cursor through the checked runner.

Items commands

Use an item

```
items.use saved "my item"
```

Use a saved object, exact name or graphic. Add on self, on last or on a scanned tile.

Equip an item

```
items.equip saved "my axe" layer "right hand"
```

Equip a saved item in a named layer, such as right hand or cloak.

Find items

```
items.find graphic 0x1BDD in backpack as logs
```

Searches the open source bag, without entering nested bags. Names must match the cached item name.

Put items in a bag

```
items.move graphic 0x1BDD to saved "wood bag" amount all from backpack
```

Move all matching stacks or an exact total amount. Open both bags first. Each move must be observed before continuing.

Scanning commands

Find nearby trees

```
scan.trees within 12 as trees
```

Collects all pages around your starting position; closest tiles first. Incomplete scans stop with an explanation.

Find cave tiles

```
scan.caves within 8 as caves
```

Finds currently loaded cave tiles. This does not mine them.

Find a tile graphic

```
scan.tiles graphic 0x0CCA within 8 as tiles
```

Use graphic or name to narrow the scan. A custom shard may use different graphics.

Find windows

```
gumps.find "craft" as windows
```

Search visible gump text or a graphic. This is read-only; use gumps.wait and gumps.press to reply.

Show found results

```
scan.trees within 8 as trees
show trees
```

Shows up to twelve loaded results, including names, graphics and coordinates. Window results include visible button details.

Walking commands

Walk to a location

```
walk.to 100 100 z 0 within 1 timeout 10 seconds
```

Nearby loaded locations only (32 tiles per move). The script waits for observed arrival.

Run to a location

```
run.to 100 100 z 0 within 1
```

Running version of walk.to. You can also use a scanned tile's loop name.

Stop walking

```
walk.stop
```

Stops assistant pathfinding.

Windows commands

Wait for a window

```
gumps.wait "craft" timeout 5 seconds
```

Match text or server serial. Exactly one complete matching server window must be open.

Press a button

```
gumps.press button 1
```

Follows gumps.wait. A quoted exact button label works when the client supplies it; otherwise choose its button number.

Wait for a menu

```
menus.wait "choose" timeout 5 seconds
```

Waits for one matching server menu.

Choose a menu option

```
menus.choose 1
```

Follows menus.wait. A quoted exact option label is also accepted.

Answer a prompt

```
prompt.reply "My answer"
```

Waits for a prompt and submits this text.

Answer a text entry

```
entry.reply "My answer"
```

Waits for a text entry window and submits this text.

Journal commands

Start fresh messages

```
journal.clear
```

Marks the current point in your journal; does not erase the game's journal.

Wait for a message

```
journal.wait "You put some" timeout 10 seconds
```

Only new system messages count, beginning at the script start or journal.clear.

Basics commands

Wait

```
wait 1 second
```

Use seconds or ms. Each wait may be at most 30 seconds.

Show a message

```
log "Finished"
```

Shows a message in script output. This does not speak in game chat.

Repeat steps

```
repeat 3 times
    log "Hello"
    wait 1 second
end
```

Repeat between 1 and 100 times. All scripts also have a run-time limit.

Check your health

```
if player.health below 50 percent
    player.cast "Heal" on self
end
```

Also supports mana, stamina, weight, items.have, journal.contains, player.dead and player.mounted.

Visit found trees

```
scan.trees within 8 as trees
for each tree in trees
    walk.to tree within 2
end
```

Runs the steps for each found tile. Use break to finish a loop, continue to skip its current item, and stop to finish the script.

Stop the script

```
stop
```

Finishes the script immediately; the Stop button can also cancel a running script.

Gathering commands

Chop nearby trees (Invicta)

```
trees.chop nearby using "lumberAxe" within 12
```

Invicta recipe: save lumberAxe, equip it, dismount and choose a 30-minute run limit. Stops for damage, weight or unknown replies; reports skipped trees. Other shards may use different messages.

1. Start here

This manual describes UO Assistant 0.7.0. The game is a complete GPU client. The assistant opens in a separate window and runs its interface and scripts away from the game rendering loop. Closing the assistant closes its paired game client; closing that client ends the same pair. Keep the assistant open while playing.

To create a script, open the assistant and search for JavaScript editor. Add a JavaScript script, enter a name and run timeout, then edit its source. Use Validate source before saving. The Macros & scripts page retains the recorder and existing task controls. Run once before assigning a hotkey. Stop cancels that run and its owned waits. A stopped or refused action is recorded in the trace.

Start with the read-only example below. Run it after entering the world. Script output appears in the assistant log; it does not send speech to the shard. Every example file in SDK/examples is a separate complete script. Examples use currently loaded facts and never need your account password.

The built-in editor supplies API suggestions and JavaScript parsing. The SDK/assistant.d.ts file supplies optional external editor types. Type declarations are editor help, not a module to import. JavaScript is the executable scripting language; the recorder and visual macro builder provide simpler alternatives.

01-status.js

Read the player and performance snapshot without acting.

```
const s = await assistant.snapshot();
await assistant.log(s.InGame
  ? `${s.Player.Name}: ${s.Player.Hits}/${s.Player.HitsMax} HP`
  : "Enter the world before running gameplay scripts.");
await assistant.log(`Draw FPS ${s.Performance.DrawFps}`);
```

2. Shard setup and personal data

In the launcher, choose the shard client folder or its client.exe. The launcher reads the version and existing Login.cfg endpoint when unambiguous, then lets you edit the host, port, encryption and graphics settings. Use the shard supplied data files, including its maps and artwork. A server connection and matching game data are different requirements.

Use Check setup before launch. Its file checks establish presence, not a guarantee that every asset is correct. Test connection makes one TCP connection to the chosen host and port without logging in or sending a game packet. A successful test proves reachability only. Use the encryption mode required by that shard; do not guess from a different server.

Each shard connection has its own private settings directory. Copy connection setup creates a new profile identity instead of copying authentication, characters or world state. Sign in manually. The public examples and installer contain no account data. Rule backups and diagnostic exports belong to your private profile unless you choose to share them.

The client accepts normal shard assistant restrictions. A script cannot grant a feature the server disables. If an action is refused, inspect the specific reason, the active target, and whether the required item or dialog is loaded.

3. JavaScript essentials

Use const for a value that will not be reassigned and let for a loop counter or changing value. Strings use single or double quotes; template strings use backticks and ${expression}. JavaScript names are case sensitive. Snapshot fields are PascalCase: Player.Hits, Items and InGame. Host API methods use camelCase: snapshot, useObject and waitForTarget.

An assistant method returns a promise. Await every host call in the order it should happen. Top-level await is supported because your script runs inside an asynchronous function. return ends that script. Use if to guard an action and a bounded for loop to repeat it. Avoid overlapping host calls with Promise.all when order, targets or dialogs matter.

Hexadecimal numbers such as 0x0E21 are ordinary numbers. Item graphics, hues and object serials are different identifiers. A graphic describes an item type; a serial identifies one object in this session. An item may disappear, move or be replaced between snapshots. Never assume a serial copied from another shard identifies the intended object.

JavaScript has no reliable exact integer representation above 2^53-1. Some dialog tokens, revisions and feature masks originate as 64-bit values. Treat those snapshot fields as informational. Dialog reply methods retain exact identities inside the host; do not copy a numeric token into a hand-built request.

02-bounded-monitor.js

Take three read-only samples with a pause between them.

```
for (let i = 0; i < 3; i++) {
  const s = await assistant.snapshot();
  if (!s.InGame || !s.Connected) break;
  const percent = s.Player.HitsMax > 0
    ? Math.round(100 * s.Player.Hits / s.Player.HitsMax) : 0;
  await assistant.log(`Sample ${i + 1}: ${percent}% HP`);
  await assistant.sleep(250);
}
```

4. Snapshots and freshness

snapshot() returns a copy of the currently loaded world. Connected and InGame distinguish the transport from an active character. SessionId and WorldEpoch change across connections and world transitions. Sequence orders snapshots and CapturedUtc is capture time. Do not keep a snapshot as if it were a live object: call snapshot() again after a wait or action.

Player holds the character, backpack, coordinates, facet, health, stamina, mana, weight, death and mount facts. Items contains loaded objects and Mobiles contains loaded nearby characters. InventoryTruncated, MobilesTruncated and Features.ItemDetailsTruncated signal incomplete lists. A missing object in a partial snapshot is not evidence that it does not exist.

OwnedInventory, Accessible, Movable and IsContainer qualify inventory objects. Equipment, another character's belongings, the bank, a closed container and nearby ground objects must not be treated as interchangeable. PropertiesComplete indicates whether item properties are complete enough for a property rule. An incomplete property list cannot prove a negative match.

Performance contains DrawFps, FrameP95Ms and ClientCpuPercent. Features.NativeTools supplies sampled performance, map and hotkey status in this version; consult the shipped SDK for its exact fields. These are measurements, not proof that every visible movement frame was newly simulated. Use the frame-time graph as well as FPS.

5. Aliases, friends and reusable scripts

Create a named alias in the assistant by selecting the intended object in the current world. SerialOrAlias arguments accept either a numeric serial or a string naming that saved alias. alias(name) resolves the alias; unresolved or stale selections are refused by actions. Use short descriptive names such as supplyBag, craftingTool or pet.

Built-in self and backpack references are resolved from the current character by the host. Use the alias selector to confirm a custom alias after changing shards or characters. Friends are stored selections; isFriend checks the saved list. It does not infer party membership, ownership, consent or hostility.

Avoid exporting personal serials in scripts you intend to share. Keep scripts generic by using aliases, actual action catalog names and per-shard rules. The examples below require no custom alias unless their description explicitly says so.

03-alias-and-friends.js

Resolve the current character and inspect the friend list result.

```
const me = await assistant.alias("self");
const friend = await assistant.isFriend(me);
await assistant.log(`Self serial ${me}; saved friend: ${friend}`);
```

6. Action results and refusal handling

Gameplay actions return an AssistantResult with RequestId, Accepted, Message and Serial. Accepted means the client accepted a request. It does not mean the shard acknowledged a successful heal, purchase, move, cast or dismount. Observe a later relevant state or journal event if success matters.

A host refusal can throw an error and ends an uncaught script. A returned unsuccessful result should also stop the dependent steps. Wrap only the part of the script that can recover meaningfully in try/catch. Log error.message and return instead of silently retrying a blocked command. Repeated requests can hide the original cause and add latency.

Typical refusal causes are a disconnected world, an unavailable spell or skill, server policy, changed item facts, a manual target cursor, a replaced dialog, stale vendor offers, incomplete inventory and an exhausted run budget. The target status and trace show the last known reason. Stop All is useful when diagnosing an unexpected sequence.

04-open-backpack.js

Request opening the backpack and report the actual request result.

```
try {
  const result = await assistant.openBackpack();
  await assistant.log(`${result.Accepted}: ${result.Message}`);
} catch (error) {
  await assistant.log(`Backpack request stopped: ${error.message}`);
}
```

7. Spells, skills and the action catalog

ActionsAvailable lists the current client's Kind, Id, Name and Group values. Find an action there rather than assuming a number from an Orion, Razor or ClassicAssist script. castSpell(id) and useSkill(id) request actual catalog entries; the native client applies availability, policy and current-world checks.

Casting a targeted spell and replying to its target are separate operations. Await the cast, wait for the new target, then reply using a target method. A manual cursor takes priority and can invalidate the script's lease. Do not steal or continually cancel a cursor the player created.

The action list is also the source for searchable spell and task hotkeys. A key binding to a catalog action avoids a script when you only need one spell or task. Use a script for a conditional sequence, an explicit wait, logging or a small bounded workflow.

05-action-catalog.js

List available actions without casting or using a skill.

```
const s = await assistant.snapshot();
const actions = s.ActionsAvailable.slice(0, 12);
for (const action of actions) {
  await assistant.log(`${action.Group}: ${action.Name} (${action.Kind} ${action.Id})`);
}
```

06-heal-self.js

Find the ordinary Heal spell, request it, and target self only after a fresh cursor.

```
const s = await assistant.snapshot();
if (!s.InGame || s.Player.Dead) return;
const heal = s.ActionsAvailable.find(a =>
  a.Kind === "CastSpell" && a.Name.toLowerCase() === "heal");
if (!heal) { await assistant.log("Heal is not in this shard's action catalog."); return; }
const result = await assistant.castSpell(heal.Id);
if (!result.Accepted) { await assistant.log(result.Message); return; }
if (await assistant.waitForTarget(3000)) {
  await assistant.targetSelf();
} else {
  await assistant.log("No new target arrived; no target reply sent.");
}
```

8. Targets and manual control

waitForTarget(timeoutMilliseconds) waits for and leases a fresh cursor for this run. It returns false when none arrives in time. targetSelf, targetLast, target(alias), targetGround and cancelTarget operate through the host target rules. A replacement cursor, manual intervention, reconnect or world transition invalidates ownership.

targetGround(x,y,z,graphic) uses absolute shard coordinates, not pixels from the game window. X and Y are 0..65535, Z is -128..127 and optional graphic is 0..65535. Facet and visibility still matter. targetLast uses the last target recorded by the native client and remains subject to current target validity.

The assistant shows whether a cursor is active, its kind and the last refusal. If a script stops at targeting, inspect that display. Backpack double-clicks, gump dragging, normal movement and game targeting continue through the native GPU client. Assistant commands should not intercept normal text entry or replace the player's manual cursor.

9. Inventory counts and item movement

count(graphic,hue,container) reports Amount, Complete and Message. Hue null means any hue; omitting the container uses the current backpack. Complete is essential when deciding whether supplies are absent. Loaded inventory may exclude unopened containers. The counter view and rule previews expose the same uncertainty.

useType selects a currently loaded matching owned item. useObject uses an exact serial or alias. moveItem requires an accessible movable loaded non-container stack, enough quantity and an accessible loaded container destination. It pins current item facts and rechecks them before dispatch. Opening a container or setting an alias does not grant access to someone else's inventory.

Do not drag with the mouse while an item workflow is in progress. A held item is a manual action and may cause refusal. Use organizer/restock previews for repeated transfers, specify maximum quantities and examine the planned source and destination before running them.

07-count-bandages.js

Count the conventional bandage graphic and report completeness.

```
const count = await assistant.count(0x0E21, null);
await assistant.log(`Loaded bandages: ${count.Amount}; complete: ${count.Complete}`);
if (!count.Complete) await assistant.log(count.Message);
```

08-move-one-item.js

Move one loaded item into a selected destination alias named supplyBag.

```
const s = await assistant.snapshot();
if (!s.InGame || s.InventoryTruncated || s.HeldItem !== 0) return;
const destination = await assistant.alias("supplyBag");
const item = s.Items.find(i => i.Container === s.Player.Backpack &&
  i.OwnedInventory && i.Accessible && i.Movable && !i.IsContainer && i.Amount > 0);
if (!item) { await assistant.log("No eligible backpack stack is loaded."); return; }
const result = await assistant.moveItem(item.Serial, destination, 1);
await assistant.log(result.Message);
```

10. Journal waits and observation

waitForJournal(text,timeout,afterSequence) matches journal text after a sequence boundary. Capture the greatest current journal sequence before sending the action you want to observe. Without that boundary, a previously loaded matching line can satisfy a wait. Text is a substring and may vary with shard language or scripts.

A journal line is an observation, not automatically proof of a particular item or spell outcome. Different actions can produce the same phrase. Prefer a unique server message and verify relevant state afterward. Do not use a fixed sleep as the only success criterion for a purchase, recipe or heal.

Use small waits in bounded loops. sleep pauses the script without blocking rendering. It is not a real-time scheduling guarantee. Local load, the client snapshot interval and server latency can make a response arrive later than the requested pause.

09-journal-boundary.js

Wait briefly for a new matching message; no game action is sent.

```
const s = await assistant.snapshot();
const after = s.Journal.reduce((last, line) => Math.max(last, line.Sequence), 0);
const observed = await assistant.waitForJournal("finished", 250, after);
await assistant.log(observed ? "A new matching line arrived." : "No new matching line.");
```

11. Gumps, menus, text entry and prompts

waitForGump(serverSerial,timeout) returns a matching currently open or newly arrived gump, or false. A zero filter accepts any current matching gump; use a specific observed server identity when you know it. It retains the exact dialog lease internally. Inspect Text, Choices, Switches, Entries and Truncated before deciding to reply.

replyGump(button,switches,entries) sends one reply through that lease. Button is a numeric button ID, not a screen coordinate. Entries are objects with Id and Text. A visible label is not necessarily the button ID. Use the dialog inspector and trace to identify a supported choice. Truncated or replaced dialogs must be re-observed.

waitForMenu(text,timeout) and replyMenu(choice) work with classic menus. waitForTextEntry and replyTextEntry handle a text-entry dialog. waitForPrompt and replyPrompt handle a current server prompt. A dialog reply is not a general keyboard typing function. Reply text is limited to 239 characters, with additional dialog-specific bounds.

Wait immediately before replying. A user response or a newly arrived identical-looking dialog invalidates the earlier lease. Do not loop the same reply against a saved token. Crafting plans require a newly observed exact gump at each reply step instead of replaying a stale button sequence.

10-inspect-gump.js

Inspect an open gump without replying to it.

```
const dialog = await assistant.waitForGump(0, 250);
if (!dialog) { await assistant.log("No gump observed."); return; }
await assistant.log(`Gump ${dialog.ServerSerial}; truncated: ${dialog.Truncated}`);
for (const text of dialog.Text.slice(0, 4)) await assistant.log(text);
```

12. Properties, buffs and matching rules

properties(serial) returns a currently cached ItemDetail or false. It does not force an object-property network query. Read PropertiesComplete before using text or numeric properties to select an item. PropertyRevision changes when the observed details change.

buffs() returns loaded buff facts: Id, Graphic, Text and RemainingMilliseconds. A buff list may not expose a shard's custom effect. It does not include every possible internal server cooldown. A missing buff cannot prove a potion is ready.

Rules can combine property predicates with AND or OR groups and assign priorities. Preview explanations identify incomplete data and why a candidate matched or failed. Numeric property recognition depends on the observed property text; use the exact form shown by your shard. Do not assume translated names share numeric syntax.

11-buffs-and-properties.js

Read buffs and one loaded item property record.

```
const buffs = await assistant.buffs();
await assistant.log(`Loaded buffs: ${buffs.length}`);
const s = await assistant.snapshot();
const item = s.Items.find(i => i.OwnedInventory && i.Accessible);
if (item) {
  const detail = await assistant.properties(item.Serial);
  await assistant.log(detail
    ? `Properties complete: ${detail.PropertiesComplete}`
    : "No properties loaded.");
}
```

13. Loot, scavenging, vendors and equipment

The loot and scavenge APIs take a rule object, preview current candidates internally, then send bounded item requests. They are active operations, not read-only previews. The return value Requested counts requests; it is not a server-confirmed number of items acquired. Use a selected destination, range, amount, delay and total run limit.

vendorBuy and vendorSell likewise build a plan from an observed vendor list and execute its requests. Although their return type is named AssistantPreview, these methods are active. Set MaximumTotal and price/quantity bounds. ManualSelection, truncated offers and changed vendor identities prevent unsafe replay. KeepAmount protects a quantity when selling.

Vendor rules use Match with Graphic and optional Hue, DesiredAmount, optional MaximumUnitPrice, KeepAmount and Properties. A bare overlapping graphic selector can be ambiguous and is refused. Property-qualified priority rules can select distinct candidates without counting one offer twice. Inspect the UI preview before building a script for the same rule.

Use the equipment page for saved dress sets, layer conflicts and two-handed checks. Use organizer and restock pages for repeated container plans. These saved rule pages are not all exposed as separate JavaScript methods. A script must use the documented API rather than inventing calls such as assistant.restock or assistant.dress.

14. Travel, combat and local render options

walkTo uses the native pathfinder over currently loaded nearby terrain. It does not promise a route across an unloaded map, locked building, blocked mobile or a shard-specific teleporter. Its stopping range is 0..3 and its timeout is 1..30000 ms. stopWalking ends assistant movement; stopping a script also cancels that run's movement.

attack, warMode, primaryAbility, secondaryAbility and toggleFlight request native combat actions. Available abilities, flight eligibility, current target and server policy still apply. These calls do not calculate combat decisions or infer friendly consent. Check the relevant loaded mobile and saved friend selection before an attack routine.

setRenderOptions accepts a complete options value. Omitted fields use defaults, so treat it as a full replacement rather than a patch. Defaults are CustomLight false, LightLevel 0, Weather true, Sound true, SoundVolume 100, Music true, MusicVolume 100, Footsteps true, HideVegetation false, TreeStumps false and DeathScreen true. Preserve a copied full current Rendering value when changing only one field.

mapWaypoint(x,y,name) sets the native map's temporary destination. It is not a persistent marker-library insertion. Use the map tools page to save a permanent categorized marker.

12-map-current-location.js

Open the map and set a temporary waypoint at the current position.

```
const s = await assistant.snapshot();
if (!s.InGame) return;
await assistant.openMap();
await assistant.mapWaypoint(s.Player.X, s.Player.Y, "Current location");
```

15. Potion cooldowns and bandage timing

Cooldown definitions belong to the shard profile. Configure a label, a tracked item graphic/hue or action, a duration and a group. Grouped entries share a ready boundary where configured. Ready bars show remaining local time and optional ready alerts. Custom durations let a shard use different potion, consumable and ability rules.

Observed timing suggestions use eligible observed use and journal facts. They are suggestions for explicit review, not automatic edits to your rules. Missing packets, a late capture, a reconnect or a gap in observed journal history can make a timing unusable. A local ready bar is an estimate and the server remains authoritative.

Bandage healing can use a fixed delay or the editable dexterity formula: clamp(base milliseconds - dexterity * milliseconds per dexterity, minimum, maximum). The formula requires a known dexterity value; unknown dexterity is refused rather than guessed. Choose self or a selected pet serial as the healing target. Recheck the selected pet when changing characters.

The UI exposes these timings directly. There is no JavaScript startCooldown, bandageHeal or editTimer method in this release. To script a bandage use, useType the shard's actual bandage graphic, wait for its fresh target and reply to your selected target. Set a conservative timeout and observe the shard response before repeating.

16. Hotkeys for spells and tasks

The hotkey page searches the client's action catalog and saved macros/scripts. Capture a key combination, inspect the action name, and save it to the active shard rules. The assistant normalizes modifiers and reports duplicate, reserved and native macro collisions. Resolve a collision explicitly instead of relying on whichever handler runs first.

A master enable switch disables assistant hotkeys without deleting them. Pass-through lets an accepted binding also reach the native input path where configured. Text and chat entry retain priority: a binding should not cast a spell while you are typing. Test a new combination with the chat field open and closed before using it in play.

Use a direct spell binding for one cast, a macro for a repeatable recorded sequence, and a script when you need conditional logic or observation. Stop controls and manual cursor priority remain in effect. A hotkey does not bypass the action queue, target leases, rule validation or shard restrictions.

The design uses the useful command trees, key capture and master/pass-through controls found in the supplied assistants. Existing Orion, Razor, Injection and ClassicAssist scripts are different languages and APIs; a similar feature name does not make their source directly executable here.

17. Recorder and visual macro builder

Record a short manual sequence, stop recording, name it and inspect each captured step. Recording does not mean every UI interaction can be reproduced safely. Save only intended actions, and confirm item aliases and dialog choices. Keep the original recorded macro separate while experimenting with a visual program.

The visual builder supports action blocks, conditions and bounded repeat blocks. Reorder blocks using the supplied controls and validate the complete program before running. Conditions include health, mana, stamina and weight percentages, loaded item counts, target activity and journal text. Choose explicit comparison values.

Program validation limits nesting, expansion and repetitions. The runtime rechecks conditions against current snapshots. Loops have finite repeat counts; they are not unattended endless automation. A manual target or a replaced dialog can stop a block even if an earlier condition was true.

A normal recorded macro allows 1..100 repetitions, at most 256 recorded steps and at most 2048 expanded actions. Visual programs apply their own validated expansion bounds too. Break a long workflow into named tasks rather than attempting to evade a limit with nested repetitions.

18. Profiles, layout and backups

Play-style profiles can capture selected rules, hotkeys and assistant layout. Use them for a healer, gatherer or crafter setup. Switching a profile changes saved definitions; it does not start playback or sign in. A profile is not a copy of the server account or a world save.

The assistant window supports navigation search and favourites, compact dashboard, size and text scaling, contrast and colour-labelled states. Save a useful layout at the display size you actually use. Placement recovery brings a saved off-screen window back into a visible working area. Docking remains a separate window beside the game.

Automatic rule backups rotate a configured count of saved definitions. The backup list and restore preview let you inspect a previous version before replacing current rules. Restore affects assistant rules, not shard files or game world saves. Save conflicts are refused when another instance changed the same definitions; reload and review rather than overwriting blindly.

Export rules for transfer only after reviewing their aliases, paths and marker labels. Personal rule exports may contain information you entered. Public installer examples are generic. Update recovery backs up assistant definition files separately before starting an explicitly chosen installer.

19. Passive alerts and bounded crafting

Passive alerts monitor low health, low mana, high weight, low supplies, durability or disconnection using available facts. Choose thresholds, quiet intervals and optional sounds. An alert reports a condition; it does not send an item use, spell or chat message. Supply and durability alerts require complete relevant observations.

A crafting plan names a selected owned tool, ingredient minima, a maximum attempt count, delays, exact gump IDs and reply buttons, and success/failure journal messages. The plan checks ingredients before each leased reply and requires a newly observed exact gump for each reply step. It does not click arbitrary screen coordinates.

Success/failure messages must match your shard's actual system messages. If the observation window expires, the plan reports an unknown result and does not retry blindly. Stop-on-failure stops further attempts when the observed failure phrase appears. Ingredient presence is not proof that a particular recipe is available.

Crafting plans run through their coordinator/UI in this release. There is no assistant.craft JavaScript method. For a custom script, build a small explicit useObject/waitForGump/replyGump/waitForJournal sequence and retain the same bounded, fresh-dialog and outcome checks.

20. Mapping and UO Mapper compatibility

The supplied ultima-mapper archive is a mapper for Ultima IV. Its dungeon and WORLD.MAP formats are not Ultima Online client data. This version instead integrates the existing UO GPU world map with a private searchable marker library, categories, visibility filters and current-position markers.

Marker tools support the recognized UOAM .map, Ultima Mapper XML and CSV formats. Imports are bounded, validate coordinates/facets and disable XML DTD processing. Review the import result and category before saving. Export writes selected marker data only; it does not convert or redistribute the shard's terrain artwork.

ClassicUO marker CSV fields are x,y,map,name,icon,color,zoom. Ultima Mapper XML uses Marker elements with X, Y, Name, Facet and Icon attributes. UOAM marker lines use a signed icon prefix with coordinates and a facet. Use the supplied export to produce a valid file instead of guessing quoting rules for commas or special characters.

Route preview uses loaded nearby terrain. A preview is informational and does not move your character. Persistent map markers and temporary mapWaypoint destinations are separate. This implementation does not embed the third-party Ultima Mapper executable, connect its external sharing service or reproduce its proprietary interface.

21. Performance and responsive gameplay

The client GPU renderer owns the whole game window. Scripts, rule editing, map imports, diagnostics and assistant navigation should not become frame-by-frame game work. Native snapshots and timing samples are bounded, and background monitoring is sampled rather than continuously scanning every object.

The performance page displays Draw FPS and recent frame-time samples alongside client process CPU, memory and native ping. Stage samples help distinguish update/render work from total frame pacing. GPU and VRAM measurements are optional OS observations; unsupported counters show unavailable instead of a fabricated zero.

60 FPS corresponds to about 16.7 ms per displayed frame. A high average FPS can coexist with visible stutters; compare the frame-time graph and p95 value while running, dragging gumps and casting. Ping measures network response rather than graphics speed. A loaded disk or asset decode can affect a frame without a high steady CPU percentage.

For a useful test, start with the shard's correct assets and default 60 FPS with VSync. Walk and run through a repeatable area, use the backpack and target a world object. Then enable one additional agent or script at a time. Keep loops bounded, avoid logging every frame and use waits between observations. Diagnostic export is opt-in and intended to omit credentials, chat text and personal paths; inspect any file before sharing it.

22. Debugging and trace navigation

Validate catches JavaScript syntax errors before execution. A parse error identifies a user source line and column; fix it before investigating the game. API completion is derived from the supported host methods. Suggestions do not imply that arbitrary browser, Node.js or .NET APIs are available.

Runtime trace records action boundaries, request results and cancellation/refusal. The editor links to Script tools for that trace; parser errors navigate to the source line. Runtime operation entries do not have JavaScript source maps or arbitrary statement stepping in this release. Match the first failure to the current target, dialog, inventory or connection state. Subsequent steps often never ran, so fixing their parameters will not solve the first failure.

Use log for concise facts and branch decisions. Log the completeness flag along with a count, or the dialog identity along with a reply choice. Avoid passwords, private journal content, account details and long object dumps. A trace exported for another person can reveal your chosen labels or workflow.

If an item action fails, open the container and inspect ownership/access. If a target fails, check whether a manual cursor replaced it. If a vendor action fails, reopen the vendor list and preview again. If a script times out, reduce its work and observe a specific event rather than increasing every timeout.

23. Runtime budgets and supported environment

Each JavaScript run uses its own Jint engine. The language is JavaScript with a deliberately small host API. There is no CLR reflection, file access, process launch, network fetch, module import, require, document/window browser DOM or arbitrary native code. Use the assistant UI for importing files and exporting markers.

A script source is limited to 65536 characters. A configured run lasts 1..1800 seconds. Runs up to 300 seconds have a statement budget of 250000, 16 MiB engine memory and recursion limit 64. Runs up to 300 seconds permit at most 1024 host operations. Longer runs scale the statement, host-call and action budgets by ceil(seconds / 300); see the scanning chapter. Host arguments are bounded JSON with at most six arguments and a 4096-byte request limit. Snapshots are capped at 1 MiB.

sleep and wait timeouts are generally 0..30000 ms. walkTo uses 1..30000 ms. Journal search text is limited to 256 characters and log text to 512. Queue delay is 100..30000 ms. These limits bound work and aid cancellation; they are not suitable for a perpetual background bot.

Stop, disconnect, a changed world identity or an expired run invalidates owned operations. Fire-and-forget calls are still observed by the worker during shutdown, but writing every intended call with await makes ordering understandable and avoids an action outliving the line you expected to control it.

24. Complete host API reference

The following is the complete public assistant method set for this release. All methods return promises and should be awaited. Methods that act return AssistantResult unless another return value is stated. Numeric range checks are additional to current-world and server-policy checks.

snapshot() Return the currently loaded AssistantSnapshot; read-only. Refresh after waits and actions.

alias(name) Resolve a saved or supported built-in alias to a serial; confirm custom selections per character.

isFriend(serialOrAlias) Return whether the serial is in the saved friend list; not a hostility or party test.

count(graphic, hue = null, container = backpack) Return Name, Amount, Complete, Message for loaded matching items. Graphic/hue: 0..65535.

log(text) Write up to 512 characters to the assistant log; does not send speech.

sleep(milliseconds) Yield this script for 0..30000 ms, cancellable and subject to the overall run timeout.

waitForTarget(timeout) Wait 0..30000 ms for a fresh run-owned target; return true or false.

waitForJournal(text, timeout, afterSequence = null) Wait for matching journal text, optionally after an explicit sequence boundary. Text: up to 256 characters.

openBackpack() Request the current player backpack gump.

openMap() Open the native world map.

dismount() Request dismount through the native client; observe Mounted afterward.

useObject(serialOrAlias) Request use of a currently valid object.

useType(graphic, hue = null) Use a matching loaded owned item type, subject to current availability.

castSpell(id) Request an available spell catalog ID; host numeric bound 1..10000, native availability still required.

useSkill(id) Request an available skill catalog ID; host numeric bound 0..1000.

targetSelf() Reply to the run-owned target with the current player.

targetLast() Reply using the current valid native last target.

target(serialOrAlias) Reply with the selected current entity serial.

cancelTarget() Cancel a permitted current target without taking over a manual cursor.

targetGround(x, y, z, graphic = 0) Reply with absolute ground coordinates and optional graphic, subject to facet/target rules.

equipItem(serialOrAlias, layer) Equip a proved loaded inventory item on an available wearable layer, awaiting the observed change. Returns boolean.

moveItem(serialOrAlias, destination, amount) Request 1..65535 units from a loaded eligible non-container stack to a loaded accessible container.

properties(serialOrAlias) Return cached AssistantItemDetail or false; never force a server property query.

buffs() Return the currently loaded AssistantBuff array.

waitForGump(serverSerial = 0, timeout = 5000) Return an open/new matching dialog or false; retain exact lease for replyGump.

waitForMenu(text = "", timeout = 5000) Return a matching classic menu or false; retain lease for replyMenu.

waitForTextEntry(timeout = 5000) Return a text-entry dialog or false; retain lease for replyTextEntry.

waitForPrompt(timeout = 5000) Return a prompt or false; retain exact prompt identity internally.

replyGump(button, switches = [], entries = []) Reply once to the retained exact gump. Entries: {Id, Text}; button IDs are not coordinates.

replyMenu(choice) Reply to the retained classic menu; numeric choice 0..65535 with native dialog validation.

replyTextEntry(text, cancel = false) Reply to retained text entry; text up to 239 characters and any smaller dialog limit.

replyPrompt(text, cancel = false) Reply to retained server prompt; text up to 239 characters.

vendorBuy(rules) Plan AND SEND bounded purchase requests from the observed vendor list; returns a plan-shaped summary.

vendorSell(rules) Plan AND SEND bounded sale requests; honor KeepAmount and completeness/current-offer checks.

loot(rules) Plan and request bounded corpse loot transfers; returns {Requested}, not acknowledgement.

scavenge(rules) Plan and request eligible ground transfers using current rules/range; returns {Requested}.

walkTo(x, y, z, stoppingRange = 1, run = false, timeout = 10000) Run native nearby loaded-terrain pathfinding; stop range 0..3, timeout 1..30000. Returns true or throws.

stopWalking() Stop assistant movement through the native client.

attack(serialOrAlias) Request attack on a currently valid target.

warMode(enabled) Set requested native war/peace mode.

primaryAbility() Request the currently available primary weapon ability.

secondaryAbility() Request the currently available secondary weapon ability.

toggleFlight() Request eligible native flight toggle.

setRenderOptions(options) Replace the full local rendering/audio options value; omitted fields take defaults.

mapWaypoint(x, y, name) Set a temporary map destination, name up to 80 characters; not persistent marker storage.

25. Snapshot and rule reference

AssistantSnapshot: Schema, SessionId, WorldEpoch, Sequence, CapturedUtc, InGame, Connected, Status, Player, Performance, Target, HeldItem, Items, Mobiles, Journal, ActionsAvailable, Actions, Capabilities, InventoryTruncated, MobilesTruncated and Features. See SDK/assistant.d.ts for exact optional and new native fields.

Player: Serial, Name, Backpack, X, Y, Z, Map, Hits/HitsMax, Mana/ManaMax, Stamina/StaminaMax, Weight/WeightMax, Dead, Mounted and the observed dexterity field. Item: Serial, Graphic, Hue, Amount, Container, Layer, Name, X/Y/Z, OwnedInventory, Accessible, Movable, IsContainer.

Mobile: Serial, Name, Body, Hue, X/Y/Z, Distance, Notoriety and any observed health facts. Journal: Sequence, Time, Text, Type, Name. ActionsAvailable: Kind, Id, Name, Group. Target: Active, Revision, Id, Type, LastSerial. Action history includes origin and captured item-type facts when available.

Features: ItemDetails, ItemDetailsTruncated, Buffs, Dialogs, Vendors, Prompt, Movement, Combat, Rendering, PartyMembers, LockedFeatures, CharacterFeatures, AssistantPolicy and NativeTools. Some features are observational and cannot be assigned from a script.

ItemMatch: {Graphic, Hue?}. PropertyRule: {Name?, Minimum?, Maximum?, Contains?}. Loot rules: Name?, Source?, Destination, MaximumRange?, DelayMilliseconds?, MaximumRunSeconds?, OpenCorpses?, Rules. Each loot entry can use Match, NameContains, Properties and MaximumAmount.

Vendor rules: Name?, Vendor?, MaximumTotal?, Rules. Each vendor entry uses Match and DesiredAmount, with optional MaximumUnitPrice, KeepAmount and Properties. Rule priorities/property groups added to the saved UI schema should be created through the rule editor and preview, using SDK types for script support.

AssistantDialog: Token, Revision, LocalSerial, ServerSerial, Kind, Text, Choices, Switches, Entries, Truncated. Vendor: Token, Revision, Serial, Buy, Truncated, ManualSelection, Offers. These identity fields describe observed state; replies keep exact host leases rather than accepting arbitrary tokens.

26. Porting scripts from other assistants

Razor and Injection command scripts, Razor Enhanced and ClassicAssist Python APIs, and Orion JavaScript APIs are separate environments. Do not paste them into this editor expecting identical behavior. First identify the intent: use an item, cast, wait for a cursor, select a target, move an item or reply to a specific gump.

Map a source command to a documented method in chapter 24. Replace global serial constants with aliases and source spell numbers with ActionsAvailable lookups. Replace polling loops with a bounded wait and explicit timeout. Keep the manual player's target priority and retain fresh dialog leases.

The existing legacy import supports only its explicitly recognized subset. Inspect the import report for every skipped or unsupported instruction. An imported task is not verified until you inspect and test it. There is no general Python interpreter or guaranteed one-to-one Orion compatibility in this release.

Features from the reference assistants were adopted where they fit a responsive GPU client: searchable action hotkeys, item and property rules, agents, aliases, counters, organized maps, script tooling and shard profiles. Implementations and names differ; consult this release's API instead of documentation for another program.

27. Troubleshooting checklist

Syntax error: validate, go to the indicated source line, check braces/quotes and verify await is inside the supported script body. Unknown method: compare the name with chapter 24 and the SDK; do not assume another assistant's API exists.

No matching item: enter the world, open its container, inspect graphic/hue and ownership, refresh the snapshot, and check completeness. Target refusal: confirm a fresh cursor arrived for this run and no manual action replaced it. Gump refusal: inspect the current gump ID and await a fresh lease before replying.

Count seems wrong: the loaded inventory is incomplete, nested containers may be closed, or the container/hue filter differs. Timer seems wrong: local bars are estimates; inspect configured shard duration, group and observed event, then adjust explicitly. Path blocked: choose a nearer loaded destination and inspect the route preview.

Hotkey does nothing: check master enable, active profile, text/chat focus, reserved/native conflicts and action availability. Stutter: compare frame-time samples, temporarily stop new scripts/agents, reduce logging/import work, and repeat a movement path. Reconnect: refresh aliases and observations; a previous world's leases cannot be reused.

Update check fails: the feed is optional and must be a direct HTTPS signed release catalog. The default feed is https://www.uoassets.com/assistant/update.json. Startup and periodic checks use saved preferences; downloading and installing still require your click. You can also open a matching local signed catalog. The development installer is unsigned for Windows publisher identity; release-catalog signatures are a separate integrity mechanism.

28. Testing a script before regular use

1. Save current rules and inspect a backup. Use a test character or local shard for a new action sequence. Confirm shard rules permit the task. 2. Validate syntax. Run a read-only snapshot/count/dialog-inspection version first. 3. Add one action and inspect its result and the actual game response.

4. Add a bounded wait with a fresh sequence or target/dialog lease. 5. Test absence, incomplete inventory, manual cursor intervention and disconnect. A useful script stops clearly in these cases. 6. Add a finite repeat count only after one iteration succeeds. 7. Assign a hotkey and test chat focus and conflicts.

8. Verify movement, gump dragging and targeting stay responsive while the script runs. Compare frame-time samples before and during it. 9. Export only generic definitions and remove personal aliases or labels before sharing. 10. Recheck after a shard changes its client version, dialogs, messages, timings or assistant policy.

The included example files are exercised offline against the actual script engine with owned synthetic world facts. That checks syntax, host dispatch and bounded behavior; it does not certify a particular live shard's spell names, recipes, delays or permissions. The final gameplay acceptance test is your manual shard session.

29. Reference sources and release scope

Local reference archives: OrionUO-master, RazorEnhanced-release-1.0, ClassicAssist-develop, Injection_0.3.30.2-master, Razor-master and ClassicUO-main. The separate ultima-mapper-master README describes Ultima IV input files. Archive documents are research material, not instructions to modify user accounts or assets.

Primary mapping references: ClassicUO World Map documentation, https://github.com/ClassicUO/ClassicUO/wiki/World-Map ; Ultima Mapper project site, https://www.ultimamapper.com/ . The implemented marker readers follow the formats recognized by the UO client and validate imported data locally.

Primary feature references: Razor Enhanced hotkeys documentation, https://www.razorenhanced.net/dokuwiki/doku.php?id=enhanced_hotkey_tab ; bandage agent documentation, https://www.razorenhanced.net/dokuwiki/doku.php?id=bandage_heal_agnet . The shipped source and tests define this application's behavior.

This development release includes the executable application, a separate assistant, this guide, generic examples, SDK types and applicable notices/source-rebuild material. It does not contain Ultima Online game assets, shard accounts or a hosted team relay. The signed website update feed is configured in the launcher. It does not install or modify a live shard. Choose your installed shard data folder when setting up the launcher.

30. UO Assistant 0.7.0 workflow and scripting additions

The assistant opens in its own window. The game keeps its GPU renderer and normal controls. Saved rules belong to the selected shard profile. Recording, script playback, observers, healing and team sharing do not start merely because their definitions were saved.

This guide covers the input, inventory, scripting and patient additions. The application guide covers installation, GPU recording, map layers, expedition routes, team sharing and runebook travel. The installed JavaScript API declaration is the exact reference for script calls.

31. Input and the native action bar

Open Input & action bars to configure the game without placing the assistant over its play area.

1. Enter a gesture such as Ctrl+Mouse4, Alt+WheelUp or a keyboard key with modifiers. 2. Select its action and numeric spell/skill/preset ID. For a saved task, enter its exact saved name. 3. Choose Game, Targeting or Chat context. Use a modified key for shortcuts intended while typing. 4. Choose Tap, Release, or Hold. Hold means one action after 350 milliseconds, not a repeating action. 5. Enable pass-through only when the same input should also reach ordinary game controls. 6. Save. The connected game receives the profile's finite configuration. A rich binding takes precedence over a duplicate legacy assistant binding while still respecting ordinary input pass-through.

Mouse left/right and essential game keys remain reserved where required by the native input handler. A refused binding is not an authorization to override ordinary game input. The game reports configuration feedback and retains the previous valid configuration on refusal.

Target presets support nearest, next, previous and lowest-known-health selection. Configure the candidate category, range, friend/controlled-pet exclusions and optional poisoned-only/known-health conditions. A target response still requires the current real target cursor. A remembered friendly or enemy selection is not authority to answer a replacement cursor.

Action buttons can run spells, skills, inventory uses, macros, scripts, equipment or travel presets, recording/replay controls, and targeting presets. Set the button label, action ID/object ID or saved task name. A hotkey label on a button is explanatory: create the actual input binding separately.

To display a custom timer, enter its exact saved Cooldown name on the button. The assistant sends small status updates at most twice per second when values change. These updates do not reinstall bindings or run actions. The display uses the real task-runner state and the passive cooldown model. Unknown, stale, estimated, journal-ready, failed and interrupted outcomes remain distinguishable. A countdown estimate does not prove a shard accepted the item or spell.

32. Object inspector and portable definitions

Open Object inspector, enter an object ID or use its target button, then inspect the loaded facts. Alternatively choose an item on Inventory and use Use selected inventory item. Missing properties remain unknown; the inspector does not request hidden data.

From the inspected item you can create:

- An exact saved alias. - A graphic/hue counter. - Protection for that exact object. - A named loot rule for that type/hue. - A JavaScript use-by-type script, avoiding a temporary object ID.

Inspect and save do not use, move or target the item in gameplay. Target selection itself is explicit.

33. Named rules and destination bags

Create a loot or scavenge set on the existing agent page, then use Named loot rules to edit its individual named rules.

Each rule has an optional graphic/hue, name selector, property groups, priority, maximum stack amount and destination bag. Destination 0 uses the set's default bag. A nonzero destination must still be a currently accessible owned container when evaluated and dispatched. A missing matching destination stops the plan instead of dropping the item into a different bag.

Within each property group, choose All for AND or Any for OR. The between-groups choice applies the same distinction to the groups. A property condition can have a literal prefix, optional numeric minimum/maximum and an optional literal substring. For example:

| Group Join | Conditions | | --- | --- | --- | | 1 | All | Luck at least 100; Lower Reagent Cost at least 15 | | 2 | All | Damage Increase at least 25 |

Choosing Any between those groups accepts either complete group. Higher-priority matching rules are evaluated first; one item is routed by one rule. Unloaded or incomplete required properties do not match. The explanation button tells you why a selected loaded item matches or fails.

34. Search, compare and move inventory

Search & compare inventory searches currently loaded owned containers by graphic, hue, name/property substring or container ancestry. Run Search explicitly; background updates do not destroy your selection.

Select one or two items to compare their loaded properties. Use Ctrl/Shift selection for a batch. Preview selected move creates a finite plan; Move selected items manually starts the same checked queue used by organizers. It accepts up to 128 selected movable non-container stacks in accessible owned inventory. A protected, held, missing or invalid selected item refuses the plan. Every move is checked again and its inventory change is observed before proceeding.

Native loot presentation can preserve the current setting, use traditional bags, use a grid, or show both. Ordinary bags remain available.

35. Type/property equipment selectors

Capture an ordinary equipment set, then open Equipment selectors. Each wearable layer can keep an exact object ID or use a selector consisting of type/hue, optional name and property conditions. An optional ranking property chooses the highest loaded numeric value; ties use object ID for a deterministic result.

The preview lists chosen objects or the first missing layer. Unknown properties, missing wearable-layer facts, protected objects and inaccessible items are excluded. One object cannot satisfy two layers. Existing weapon-conflict and occupied-layer checks still apply.

Set Undress bag to an accessible owned bag, or use 0 for the backpack. Replacement must still be explicitly permitted by the equipment set. The assistant does not infer a bank or external container as an undress destination.

36. Recording ordinary gameplay

Gameplay recorder records ordinary manual double-clicks, cast/skill uses, item moves/equips, target responses, supported gump/menu/prompt replies and attacks observed by the client bridge. It excludes automated playback, configuration traffic and authentication.

Start recording after entering the game. Perform a short sequence, then stop and save. Review the macro before running it. The recorder adds waits for fresh target/dialog state and removes captured session, target and dialog authority tokens. It preserves the requested choice/button and other reviewed action details rather than granting an old dialog permission to act again.

Select a recorded object use and replace it with a type/hue match, or replace an object use/entity target with a saved alias. This is useful after a stack, tool or patient receives a different object ID. Play from selected step validates the remaining block structure; it refuses a slice containing orphaned branch/repeat boundaries. Use the existing macro editor to insert or rearrange steps and complete structured blocks.

Recording a successful request does not prove shard success. Changed dialogs, ambiguous targets, incomplete inventory, missing aliases or a refusal stop playback; there is no blanket automatic retry.

37. Migrating UOSteam and Razor macros

Macro migration reads text; it never loads a legacy executable, Python interpreter, plugin or DLL. Paste a UOSteam script or open a text file. Serialized Razor actions use the existing enumerated parser first.

Select Translate only. The page shows generated JavaScript, converted-line counts and exact blockers/warnings. Any unsupported/error line prevents a runnable conversion from being saved. Review every warning before Save reviewed conversion as script. Saving does not execute it.

The converter supports useful finite combinations of:

- If/else-if/else conditions and bounded numeric/list loops. - Local aliases, list creation/push/pop/indexing, ignore lists and timers. - Loaded object/type/count/layer/range checks, loaded skills and gump text checks. - Item use, checked item moves/type moves, equipping and explicit target waits/responses. - Spell/skill catalogue lookup, gump waits/replies, journal waits and optional system-only scope. - Assistant output messages, cancellation, war mode and explicit attacks.

Important differences are reported rather than hidden:

- While loops and list iteration are capped at 100 iterations; the overall run/operation limits also apply. - Lists and timers exist only during that script run. A list holds at most 512 entries and the helper limits list count. - promptalias requires the alias to be selected and saved before running. Interactive legacy prompting is not replayed. - Head/system messages appear in assistant output; overhead placement and colour are not reproduced. - Precise container slot placement, arbitrary pixel clicking, unsupported mobile searches, unbounded replay and unknown commands require manual translation. - Skill predicates use only actual server-observed loaded skill values. Unknown values stop the script instead of being treated as zero. - Type searches operate on current loaded facts and explicit container/ground scope. A move waits for an observed inventory change and stops if it is not seen.

A read-only audit of the supplied 200 UOSteam scripts produced 32 complete conversions, compared with 2 accepted by the earlier importer. It converted 14,451 command/control lines across the corpus, including partial conversions that remain blocked. None of those supplied scripts was executed. The audit used synthetic action-name entries to measure syntax coverage, so it does not establish live spell IDs, shard compatibility or gameplay success. There is no claim of complete UOSteam, Razor Enhanced Python or Injection compatibility.

38. Source debugger

Use Source debugger & bench for actual JavaScript source breaks. Select a saved script or copy the editor source. Enter comma-separated source line numbers, starting at 1, and optionally break on the first statement. Then explicitly start Debug this source in game.

When paused, inspect the current original-source line, column, primitive local variables and function call stack. Object/array previews are deliberately shallow; inspection does not call user getters, toString or arbitrary watch expressions. The UI provides Continue, Step statement and Stop. This source debugger is separate from the existing macro/API-boundary trace.

A pause revokes the current native action grant and invalidates target/dialog leases. Resume validates the current session, issues a new grant and requires fresh leases again. Script runtime includes paused time. A long pause can therefore reach the configured deadline. Stop and window closure cancel a paused worker before later actions can run.

39. Offline script bench

Simulate against current loaded facts evaluates JavaScript in a constrained isolated engine using a copied snapshot, or an illustrative fixture when no game is connected. It records proposed API operations and script output. It does not connect to a client, send packets, access game accounts, read script-selected files, install modules or contact a network provider.

Offline target/journal checks use only the supplied snapshot. Gump/prompt waits return false. Action results are labelled simulations. Server replies, timing, changing inventory, movement and rendering still need an in-game test. A successful bench run proves neither shard acceptance nor combat safety.

40. Visual workflows and reusable functions

Visual workflows is a block editor with a connected diagram. Add actions, pauses, conditions, else/end blocks, finite repeats and calls to named reusable functions. Choose Root or a function to edit its block list. Move/remove blocks, validate, save and manually run.

The compiler expands functions into the same macro executor. It does not create another privileged scripting runtime. Recursive functions are refused. Function nesting is limited to eight calls, macro block nesting to four, and the resulting macro to 256 steps/2,048 expanded steps. Each repeat is 1-100 iterations. Ordinary current-state checks, shard policy, target leases, item proofs and Stop remain active.

41. Priority observers

Create several Priority observers over health, mana, stamina, weight, item counts, target state or literal journal conditions. Link each to a saved script, macro or visual workflow. Set a priority and quiet time; optionally require a rising edge.

Saving never arms observers. Arm observers for this character session is an explicit action. One highest-priority eligible task can enter the existing single-run action arbiter; other pending edges are retained. Held items, active targets, movement and recent manual actions take precedence. Manual game actions stop a task started by the observer scheduler. Reconnection/character changes, Stop/disarm and closing the assistant disarm observers. They do not automatically rearm on the next login.

Choose short bounded tasks for observers. A task that waits indefinitely for unrelated input prevents other automated actions from using that one action slot until it stops.

42. Patients, pets and cooldown outcomes

Patients & pets shares the saved Healing settings. Choose self, a specific server-identified controlled pet, or an explicit patient list. Add saved friends, currently loaded party members, a picked patient or an entered object ID. Patient-list selection chooses the lowest known health fraction in configured range. Missing health/poison facts stay unknown. Bandages remain limited to two tiles; a configured spell can use a longer range up to 12 tiles.

The patient chosen for a healing action stays fixed while its fresh target cursor arrives. A changed or out-of-range patient cannot redirect that cursor to somebody else. A healing watch waits, within its displayed limit, for transient missing health or range to recover before starting another action. Healing and Patients & pets share the same named patient and method settings. Timing edits fixed/dexterity bandage formulas only. Cooldowns are local estimates, not confirmation of server healing. See Healing and command panels for the simple workflow.

An explicit pet-command button speaks the configured shard command once while a nearby server-identified controlled pet is loaded. It does not infer commands from pet artwork, start automatically or assume a pet obeyed.

For a saved cooldown, configure specific failure and interruption journal messages. Only newly observed matching events can end a running estimate. Failed (journal) and Interrupted (journal) do not mean ready and do not emit a ready alert. Journal-ready completion remains distinct from an elapsed estimate. System-only observation excludes ordinary player speech; choose All only when appropriate for that shard.

43. Sound, journal and readability

Sound presets lists up to 32 recent sound events. Select an ID and use Listen once before deciding whether to mute it. Listening bypasses that individual ID mute but retains master-volume/background sound behaviour. The saved mute list is explicit and visible; unmute one ID or clear it. Music, footsteps, ordinary sound volume and other display choices remain on the existing controls.

Focused journal adds literal search/highlights, source selection and repeat folding. It changes only this reading view. The original Journal page and captured messages remain intact. Saved highlights use readable background/foreground colours. Existing Workspace settings provide font/scale, high contrast and accessible palettes across the separate assistant; role layouts can pair them with game presentation settings. These additions do not claim a new controller driver or automatic gamepad profile.

44. Template versions and optional assisted editing

Script library & edit preview records local publisher/version/description and a SHA-256 content hash for a saved script. Check whether its source still matches the saved version before sharing it. Definitions and profile backups remain explicit reviewed files; a template manifest does not execute or fetch remote code.

The included 0.3 template selector provides known-skill reporting, checked weapon-alias equipment and per-rule loot-bag examples. Adding one explicitly saves a new script and its 0.6.0 content manifest; it does not run. Review the example's required aliases and shard assumptions first.

The optional assisted-editing workflow builds a local context preview from only the pasted selected source, your requested change and installed documented API signatures. It does not automatically launch a provider, require an API key, transmit an account/profile or execute a generated proposal. Copy only after reviewing the preview, use a provider of your choice, and validate any pasted replacement. Save/run is a separate explicit action. This is an editing aid, not an autonomous gameplay agent.

45. JavaScript API additions

equipItem(serialOrAlias, layer) validates one loaded item, refuses an occupied/conflicting/protected layer and observes the equipment change. Use a saved equipment set for explicitly configured replacements and an undress bag.

waitForJournal(text, timeoutMilliseconds, afterSequence?, source?) retains its existing arguments and adds optional "All"/"System" source selection. The default remains All for existing scripts. A wait can be 0-30,000 milliseconds. The sequence fence prevents a retained older line from satisfying a new confirmation wait.

javascript

```
// Use the installed API spelling and argument order.
await assistant.equipItem('spareWeapon', 1);

const before = (await assistant.snapshot()).Journal
  .reduce((n, line) => Math.max(n, line.Sequence), 0);
const confirmed = await assistant.waitForJournal(
  'You finish applying the bandages', 15000, before, 'System'
);
await assistant.log(confirmed ? 'Observed finish message' : 'No finish observed');
```

46. Testing limits

Automated tests exercise the actual bounded JavaScript engine, source debugger pause/locals/call stack/Stop, fresh gump leases, item/rule proofs, finite graph execution, scheduler priority/manual exclusion, patient unknown-state handling, cooldown outcomes and the actual WPF save/preview handlers. Native tests separately exercise input contexts, holds, targeting, urgent deltas, layout and sound/status controls. Synthetic UI screenshots are labelled fixtures. These tests do not substitute for a new manual gameplay session on each shard, nor prove all third-party macros or custom gumps compatible.

47. Expeditions, shard layers, travel, team sharing and role layouts

These tools are in the separate UO Assistant window. They use the GPU game client's ordinary controls and loaded observations. They do not replace shard rules, movement timing, travel restrictions or the server's assistant policy. A saved setting never starts an agent or travel action on login.

48. Expedition & recovery

The assistant notices a fresh alive-to-dead transition and saves a private death record with location, facet and time. A character already dead when connecting is treated as a baseline; reconnecting does not create duplicate deaths. Up to 64 death records are retained. Add a note to the selected death or show its destination on the native map.

Save healer locations explicitly while standing at the healer. The assistant does not infer resurrection service from a mobile's artwork or name. The list sorts same-facet locations by nearby tile distance. Its compass direction and tile count describe a destination, rather than a verified traversable route.

To make a numbered expedition route:

1. Enter a route name and choose New empty route. 2. At each stopping point, enter a waypoint name and choose Append current waypoint. 3. Select a numbered waypoint to show it on the map, plan a nearby route, or explicitly walk to it. 4. Use Stop walking, the client's normal movement controls or Stop all to interrupt owned movement.

There are at most 64 saved routes and 128 waypoints per route. Removing a waypoint preserves the order of the remaining ones. These are saved directions; the assistant does not automatically walk a whole expedition route or teleport between facets.

Set return point here preserves one homeward destination. Show current short trail puts recent points on the native map. The trail has at most 256 positions, is held in memory, and resets on disconnect/session change; the displayed sample uses the most recent 128 positions. Showing the trail replaces the current assistant marker overlay, so apply your saved map layers again to restore them.

The nearby planner searches only loaded terrain inside its bounded region. Unknown tiles are not assumed walkable. The result can account for the terrain and obstacles loaded by the client; it cannot prove that an unloaded area is traversable or that a shard will accept a step. A route preview starts no walking. Doors, mobiles, changing obstacles and server corrections can invalidate any client-side plan.

49. Shard layers & coordinates

First save a Shard label and use Fingerprint map folder to choose that shard's client-data folder. This reads map/static file names, lengths and samples of their first/last 64 KiB on a background worker. It does not change game files or copy them into the assistant. The resulting sampled fingerprint identifies a map-data set; it is not a full integrity hash of every byte. After updating custom maps, compute a new binding and review the affected layers explicitly.

A versioned map layer contains a schema version, permanent layer ID, revision number, shard label, sampled map fingerprint, markers and optional polygon zones. The assistant accepts a layer only when its shard and fingerprint match this profile. An existing layer can be replaced only by a higher revision with the same ID. This prevents an old downloaded pack from silently replacing a newer one. Export and remove existing layers before changing the profile's shard/map binding; review them against the new data before importing them again.

Choose Create layer from marker library to turn points already imported on Map library & terrain preview into an exchangeable layer. That page supports compatible CSV, UOAM .map and Ultima Mapper marker XML data; no external mapper program or icon archive is required. Export a selected versioned layer as JSON to share the data you choose.

For a JSON layer import:

1. Optionally enter category mappings such as Shops=Services;Banks=Services. 2. Choose Preview layer JSON. Review marker/zone counts, duplicate counts, sample point names and whether it replaces an earlier revision. 3. Choose Apply reviewed layer to save the reviewed data privately. 4. Choose Apply visible layers to game map to update the native map overlay.

Duplicates are compared by name, category, facet and coordinates. Duplicate points are skipped in the preview; the rest are imported as a single reviewed layer. A failed import applies none of the candidate points. Toggle a layer's visibility or remove it as required. Undo last layer edit keeps up to eight earlier layer states in memory; it restores layers without reverting unrelated death records, routes, travel presets or other assistant definitions.

Each layer has at most 2,048 markers and 64 polygon zones. A polygon needs 3-128 vertices on one facet. To draw a zone, choose a layer, enter its name/category, capture the current location at each corner, then save the zone. This increments that layer's revision. The page reports which visible zones contain the character, and applied zones appear as outlines on the native map. The native overlay accepts at most 256 visible markers, 64 zones and 4,096 total polygon vertices at once; hide layers to stay within those limits.

50. Custom sextant coordinates

Do not assume a custom shard uses standard Britannia coordinates. Enter and save that facet's map width, height and origin tile. Obtain those values from the shard's map author or verify them against known points. The initial editor values are examples until explicitly saved; no standard origin is automatically assigned.

Enter latitude degrees/minutes with N/S and longitude degrees/minutes with E/W. Degrees and minutes must be finite, minutes are from 0 up to but excluding 60, latitude is at most 90 degrees and longitude at most 180 degrees. Coordinates wrap inside the saved map dimensions. Show sextant destination creates a map destination using that saved facet origin; it does not start movement.

51. Runebooks & travel

Runebook layouts vary between shards. Learn the real loaded dialog instead of copying a historical button number:

1. Pick an accessible book from your inventory, then open it through the game or Open selected book. 2. Choose its loaded gump and a captured button. Add optional required text that identifies the expected page, then choose Add selected captured button. 3. For a book with local page navigation, select its captured Open page choice and add that step first. Use Open selected local page to visit it manually, then add the actual recall/gate button from the new page. Repeat for any additional page/reply stages in their real order. Local page changes use an exact fresh dialog lease and send no server reply; the next server button must be captured at the resulting new revision. 4. Set any shard-specific minimum mana and resource quantities. Optionally require literal loaded book-property text for a charge/status check. Property text is a literal check; the assistant does not guess the shard's charge formula. 5. At a known destination, save the current location as an arrival coordinate. This allows arrival confirmation; without it, the run stops after dispatching the configured replies. 6. Name, preview and save the destination. Explicitly start it with Travel to saved destination or a bound travel hotkey.

Every stage waits for a newly observed exact gump instance/revision and checks that the configured button is present. A changed, ambiguous, incomplete or missing dialog stops the run. The book must remain loaded, accessible, unprotected and in your inventory. Missing mana/resources, an active target/held item, movement, death or a session change also stops it. The assistant never reuses a saved gump token.

Failure messages are literal system-text fragments separated by |; edit them for your shard's language and wording. A matching fizzle/blocked/charge failure stops without retry. With an arrival coordinate, the assistant observes the same character near that point for a bounded period. A cross-facet arrival can be observed after the final reply, but no further action is dispatched across that world/session fence. No observed arrival means the outcome remains unconfirmed; no second travel request is sent automatically.

The runebook editor supports 128 named destinations, 16 reply stages and 16 resource checks per destination. It has no built-in spell/charge exploit or special shard packet. Existing shard restrictions and the assistant's stop controls still apply.

52. Team map & help pings

The page first shows members supplied by the shard's party system. Loaded health/position facts are shown when known; a member outside loaded range remains unknown. The native world map can also show party positions supplied by the shard itself.

An optional authenticated relay extends voluntary sharing between participating users. It is off every time the assistant opens. The application creates no external service or cloud deployment, and makes no sharing request until you explicitly connect.

Enter the relay root address, room, public label and private sharing secret. HTTP is accepted only for loopback; other addresses require HTTPS. The secret is entered in a password field, used in memory, cleared from the field after connecting and never stored in assistant preferences. Choose whether to share known health/death status; location and your chosen label are the basic shared fields. Accounts, passwords, inventory, game chat and shard credentials are not transmitted.

Updates run on a background worker about once a second, with one request at a time and a short timeout. Slow/offline sharing cannot hold up game input or rendering. A peer older than five seconds is marked stale, and health older than 15 seconds is hidden. Help, rendezvous and marker pings expire after 30 seconds. Select a member or ping to show its last shared destination on the game map; this starts no movement or targeting. Treat shared coordinates as voluntary reports from other participants rather than server authority.

Choose Stop sharing / disconnect to end sharing immediately. It also ends when the assistant closes or the paired game exits. A temporary disconnect pauses sending until fresh character state is available; the page continues to label stale observations.

53. Hosting the optional relay

UOAssistant.TeamServer is a separate optional administrative tool. The game never launches it. It stores no accounts or files; room members and pings exist only in memory. Limits are eight rooms, 64 members per room, 128 pings per room, bounded requests, eight concurrent requests and at most two updates per second from one client ID. Inactive members expire after 60 seconds.

Give the relay a random secret of at least 32 printable characters. Use a different secret for each team/service and distribute it privately. Everyone who knows the relay secret can participate, so replace it if a member should lose access. It is a shared-secret service, not an account/permission administration system.

For a local test on Windows, open a terminal deliberately for this optional tool, navigate to its folder and run:

The relay reads the secret only from the process environment. Do not put it in command-line arguments, public scripts, screenshots or published packages. Press Ctrl+C to stop. If Windows refuses an HTTP listener, an administrator must reserve the exact chosen listener for the account or configure an HTTPS binding; the tool does not request elevation or alter firewall settings automatically.

For remote users, an administrator must provide an HTTPS endpoint and certificate, either directly on a configured Windows HTTPS listener or through an HTTPS reverse proxy to the loopback listener. Keep the underlying HTTP listener private, do not disable TLS certificate validation and do not expose an unauthenticated forwarding endpoint. This release does not deploy or configure hosting for you.

Developers can build the relay from the matching public source:

PowerShell command

```
$teamSecret = Read-Host -AsSecureString 'Team sharing secret'
$env:UO_ASSISTANT_TEAM_SECRET = [Net.NetworkCredential]::new('', $teamSecret).Password
.\UOAssistant.TeamServer.exe --listen http://127.0.0.1:9696/
Remove-Item Env:\UO_ASSISTANT_TEAM_SECRET
```

Command

```
dotnet publish src/Assistant.TeamServer/Assistant.TeamServer.csproj -c Release -r win-x64 --self-contained true
```

54. Role layouts

Use named role presets for healer, crafting, combat or exploration work. Capture current layout saves the assistant dimensions/topmost setting/current section, game client dimensions, presentation settings and identities/positions of currently open movable game windows. It does not capture container contents, accounts or remote dialog tokens.

Select a role and review its preview, then explicitly apply it while stationary and with no active assistant run. Only uniquely matching windows already open in this character session are repositioned. Missing/ambiguous windows are skipped; containers are not reopened and server dialogs are not replied to. Positions are clamped into the chosen client bounds. The assistant remains a separate window, with its own screen position kept visible.

Undo last application restores the layout captured before the last application. Undo is held in memory until the assistant closes. Neither applying nor undoing a role logs in, casts spells, starts tasks or enables an agent. Up to 32 role presets can be saved. A role is a workspace/presentation preset, rather than an automatic gameplay loadout.

55. Verification and remaining live checks

The mapping/travel/team tests use synthetic map/state fixtures and an authenticated loopback relay. They cover death transitions and stale sessions, bounded trails, custom sextant validation, layer binding/revisions/duplicates/categories/polygons, settings ownership, exact fresh travel leases, no retry after a fizzle, observation of a cross-facet arrival, relay authentication/rate/expiry, actual background HTTP exchange and sampled map fingerprinting. They do not use a real account, shard or world save.

Live shard checks still matter for the particular runebook pages/messages, charge text, custom map origins, movement/collision corrections, remote HTTPS deployment, window placement at different DPI settings and the visible native map overlays. Only use a shard's permitted assistant functions.

56. High-quality GPU video and replay

Open Recording in the separate assistant. Start video recording or Start replay buffer explicitly after entering the world. Capture selects only the paired game window by process identity and window handle. It never falls back to the desktop or another window. The microphone is always off. Game-only audio uses Windows process loopback on Windows 11; switch it off for video-only capture on older supported Windows.

Auto tries HEVC, AV1, then H.264 hardware encoders. NVIDIA, Intel and AMD paths are probed at runtime; actual hardware acceptance in this release was tested on NVIDIA. HEVC was the smaller output at comparable small-text SSIM in the controlled 2D fixture, so it is the default preference. AV1 remains selectable; H.264 is useful for older players and editing software. There is no automatic CPU encoding fallback.

Balanced at 60 FPS is the starting preset. Compact increases compression; Detailed preserves more fine detail. Maximum width downscales larger windows and does not enlarge smaller windows. Native-size capture crops an odd final pixel instead of scaling the entire scene. The NVENC bitrate cap limits average burst size but content, resolution and quality still determine file size. A lower setting is not a promise of identical visual quality.

Save recording settings applies the selected codec, quality, FPS, maximum width, bitrate, replay duration, disk budget, game audio, cursor and local clips folder. Blank clips folder uses Videos / UO Assistant. Choose a local ordinary folder with enough free space. Settings belong to this shard assistant profile. Use ToggleRecording, ToggleReplay and SaveReplay in the hotkey or action-bar editor to assign your own controls. Nothing starts automatically when loading a profile.

Replay keeps encoded two-second chunks, bounded by selected duration and disk budget. Save replay exports only complete chunks, so it can finish about two seconds before the current picture. Saving preserves the running buffer. Stop and save writes an MP4 without another encoding pass. Trimming copies compressed data near keyframe boundaries; it preserves the original and can include a little extra footage.

Minimizing or closing the paired game ends capture. Closing the assistant finalizes complete video before closing its paired game. Complete recovery segments are preserved if an encoder or finalization error occurs. Ordinary recording is limited to eight hours per session. Low disk space stops recording; final MP4 creation needs additional space close to the clip size. If finalization cannot complete, preserve the .recovery folder and free space before using FFmpeg to remux its complete segments. Never delete recovery files before verifying the resulting video.

The diagnostics page reports encoder CPU, encode FPS, drops and the actual game frame P95/P99. A controlled GPU-client UI scene held about 59.97 FPS before and during hardware AV1 recording, with P95 approximately 18.04 versus 18.11 ms and no encoded drops. This is a synthetic rendering/capture test, not a live movement or crowded-shard benchmark. Compare your normal path with recording off and on; choose 30 FPS or a smaller maximum width if GPU saturation makes frame times worse.

57. Quiet startup and private diagnostics

Normal launcher, game and assistant startup creates no command window. The optional manually started team relay is a server tool. Recording helper processes also start without a console. The game and assistant retain their separate visible windows.

Diagnostics shows Draw FPS, frame P95/P99, long frames, assistant CPU and recorder statistics. The game saves warning/error diagnostics through a bounded background queue. Detailed tracing is opt-in; it is not required for normal play. The queue holds at most 512 messages and game log files have an 8 MiB limit. The UI reports dropped diagnostic messages instead of blocking rendering.

Open current private log folder opens the current paired session log. Logs can contain local paths and server text: review them before sharing. No automatic log or video upload occurs. The publisher-unsigned development installer still verifies its own payload, the signed update catalog and exact download hashes; Windows publisher signing is a separate property.

58. Recording source, dependencies and replacement

The recording tools are an LGPL-3.0-or-later shared FFmpeg build, run as a separate process. The release includes its exact binary inventory and a separate corresponding source/rebuild kit with FFmpeg source, dependency source, build recipes, configuration and license texts. Consult Recording-Dependencies.html and the release checksums. The application does not claim ownership of FFmpeg.

A developer can rebuild or modify FFmpeg and update the media-tools.json hashes and runtime inventory in the matching public application source, then rebuild the launcher with its new runtime-manifest hash using the supplied build and recombination material. This permits a compatible modified library/tool build; do not disable checks on an existing signed public update feed. Keep the original application source and licenses when redistributing changes.

Scanning terrain, items and open gumps

UO Assistant 0.7.0 adds scanTiles, findItems and findGumps. These methods inspect loaded client facts without opening containers, asking the shard for names, loading map blocks or clicking controls. The SDK now exposes 48 methods. Three new read-only examples are available in the script example selector and SDK/examples.

Query properties are PascalCase. Unknown property names are rejected, so a misspelled filter cannot silently broaden a search. Name matching ignores case and uses substring matching unless ExactName is true. Graphic and Hue are numeric artwork and colour identifiers; a serial identifies one actual item.

Run a new search before acting on a result. Ordinary useObject, targetGround, waitForTarget and dialog-reply methods retain their existing live checks. Inspection does not create a target or dialog reply lease.

Nearby trees and cave tiles

scanTiles({ Kind: "Tree", Radius: 12 }) reads map statics around the player. Kind also accepts Cave, Land, Static and Any. Combine Name, ExactName, Graphic and Hue filters. Radius is 0-18 tiles; only currently loaded cells within the client view are inspected.

Each page returns Map, X, Y, Radius, CapturedUtc, Tiles, NextOffset, UnloadedTiles, Truncated and Complete. A tile has X/Y/Z, original Graphic, seasonal DisplayGraphic, Hue, Name, Kind, Tree, Cave and Distance. Keep the returned Map/X/Y and the same radius/filters for every page. Continue with NextOffset until it is null. Sum UnloadedTiles and retain any Truncated flag from every page before claiming a complete area scan.

Native work examines at most 32 cells per page, returns at most 256 matches and observes the existing cooperative 2 ms budget between cells. A dense cell is capped at 128 linked objects. Limits are reported explicitly. Moving or changing map can invalidate a multi-page observation; start again after moving.

Tree classification uses the client tree filter. Cave classification uses the configured static cave filter; land tiles whose tile-data name contains cave are also candidates. Custom shard graphics may need an explicit Graphic or Name query using Kind Any. Classification does not prove that the server permits chopping or mining, or that resources remain.

Finding loaded items by name or graphic

findItems({ Name: "axe", Scope: "Owned", Graphic: 0x0F49 }) combines the requested filters. Scope can be Any, Ground, Owned, Equipped or Backpack. A numeric Container restricts the search to its contents; Recursive defaults to true. Range is 0-32 and applies to Ground scope. Movable and IsContainer can further narrow results; Limit is 1-512, default 128.

The returned Items use the best known cached property name, item name or tile-data name. Complete refers to currently captured facts, not the entire shard. UnknownNames counts items with no searchable name. Missing inventory, truncated results and unopened requested containers are reported as partial. No property request is sent and unseen container contents cannot be discovered by scanning.

An item result can be used with useObject(item.Serial) after checking that it is still the intended accessible object. For static trees use the coordinates and original Graphic with targetGround after waiting for the axe cursor.

Inspecting general game windows

findGumps() includes visible local game windows and server dialogs belonging to the current world. Filter by Type, LocalSerial, ServerSerial, Name, Graphic or Hue. Name searches the window type or visible control text. Graphic and Hue identify artwork on a matching control. Follow NextOffset for further pages.

Each window includes Type, token, local/server identifiers, position, size, active page, FromServer, Controls and Truncated. Controls include their type, serial, position, size, page, enabled state, text and supported graphics/hues; buttons also expose Button, Action and ToPage. Nested visible controls are inspected to depth 12, with at most 256 visited controls per window. Hidden controls and inactive pages are excluded.

Editable fields are excluded from this general inspection. Text is limited to 256 characters and may contain HTML markup. A scan can be partial when the time, window or control budget is reached. Opening, closing or changing a window while paging can change results; rescan before acting. Replies to server gumps still require waitForGump and a current reply lease. Native windows are inspection-only through this API.

Longer bounded scripts and testing

Scripts may explicitly select a run limit from 1 to 1800 seconds. The default remains 60 seconds. At up to 300 seconds the existing budgets remain 250,000 statements, 1,024 host calls and 256 dispatched requests. Longer limits multiply these by ceil(seconds / 300), up to six times. Read-only native scans count as dispatched requests. The 16 MiB memory, recursion, source size, output and per-call bounds are unchanged.

Stop, disconnect, session changes and shard policy still end or refuse affected runs. A script must report unfinished work when it reaches its time, operation, weight or tool limit. A bounded run is not an unlimited background service.

The offline bench can search its captured item snapshot. It refuses terrain and general gump scans because those facts are absent from that snapshot. Connected-client and synthetic native tests cover those methods; do not treat an offline success as evidence of live harvesting.

This development preview has automated client, host and synthetic script checks. Live shard harvesting, custom cave graphics and populated-world scan performance still require receiving-system gameplay checks.